DATA 260 Homework 9

Priyank Mehta | SJSU ID 019152562

GitHub repository: https://github.com/priyankmehta5/data260-2562

Personal configuration

Configuration	Value
SID4	2562
PORT_BASE	8762 = 8000 + (2562 mod 900)
PREFIX	s2562
SEED	2562
VERIFY_SEED	262562
DOMAIN_ID	2
Domain	Municipal transit incidents
Processor	Intel Core i7-8750H CPU @ 2.20 GHz
Memory and operating system	15.86 GB RAM; Windows 10
Graphics	NVIDIA GeForce GTX 1050 Ti; Intel UHD Graphics 630
Local model	qwen3:1.7b
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Verification commit	a67d98af1091a348f6f4b3f0605c4d0d53bf63f0

The municipal transit application adds Redis access counters and a semantic response cache in front of the HW8 Kafka pipeline. A separate Answering Agent and Evaluator Agent use reference facts, feedback, and reusable lessons.

The initial cache test served 6 of 12 queries from cache. The five-threshold experiment measured 200 calls. Threshold 0.95 minimized the assumed cost, while still producing four false hits. All four agent variants answered the four held-out questions correctly. The live verification passed 36 checks.

Part 1 Redis access counter

increment_incident_views() uses s2562:views:incident:5007 and Redis INCR on every access. A cold key starts from the persisted count using SETNX. Every multiple of ten is written to MySQL; the conditional update prevents an older checkpoint from reducing the stored count.

```
data260-2562 > backend > hw09_database.py > ...
10   class IncidentDatabaseWithCache:
31       def increment_incident_views(self, incident_id):
32           if isinstance(incident_id, bool) or not isinstance(incident_id, int) or incident_id <= 0:
33               raise ValueError("incident_id must be a positive integer")
34           key = f"{PREFIX}:views:incident:{incident_id}"
35           # Only a cold Redis key needs a database read; SETNX avoids resetting another caller.
36           if not self.redis_client.exists(key):
37               self.redis_client.setnx(key, self.incident(incident_id)["view_count"])
38           count = self.redis_client.incr(key)
39           if count % 10 == 0:
40               with self.engine.begin() as connection:
41                   connection.execute(
42                       text(
43                           "UPDATE incidents SET view_count=CASE WHEN view_count<:count THEN :count "
44                           "ELSE view_count END WHERE id=:id"
45                       ),
46                       {"count": count, "id": incident_id},
47                   )
48               event("Synced views to database", incident_id=incident_id, count=count)
49           return count
```

Figure 1. Part 1 counter implementation in backend/hw09_database.py.

```
Priyank Mehta - HW9 live execution
>>> print("Priyank Mehta - live counter");
Priyank Mehta - live counter
>>> print("\n".join(f"Access #{r['access']}: Total={r['count']}; MySQL={r['database_count']}" for r in live["counter.jso
n"]["records"]))
Access #1: Total=26; MySQL=20
Access #2: Total=27; MySQL=20
Access #3: Total=28; MySQL=20
Access #4: Total=29; MySQL=20
Access #5: Total=30; MySQL=30
Access #6: Total=31; MySQL=30
Access #7: Total=32; MySQL=30
Access #8: Total=33; MySQL=30
Access #9: Total=34; MySQL=30
Access #10: Total=35; MySQL=30
Access #11: Total=36; MySQL=30
Access #12: Total=37; MySQL=30
Access #13: Total=38; MySQL=30
Access #14: Total=39; MySQL=30
Access #15: Total=40; MySQL=40
Access #16: Total=41; MySQL=40
Access #17: Total=42; MySQL=40
Access #18: Total=43; MySQL=40
Access #19: Total=44; MySQL=40
Access #20: Total=45; MySQL=40
Access #21: Total=46; MySQL=40
Access #22: Total=47; MySQL=40
Access #23: Total=48; MySQL=40
Access #24: Total=49; MySQL=40
Access #25: Total=50; MySQL=50
```

Figure 2. Part 1 live run with 25 additional accesses and MySQL checkpoints.

The counter continued from 25 rather than resetting. Redis reached 50; the database synchronized at 30, 40, and 50. Between successful checkpoints, losing Redis can lose up to nine recent views.

Part 1 Deployment and database persistence

Redis Stack provides the counter and vector-search index. Kafka, Planner, Writer, Reviewer, and the incident worker run as Docker services. FastAPI runs on the host at port 8762; its health response shows HW9 enabled and the selected threshold of 0.95.

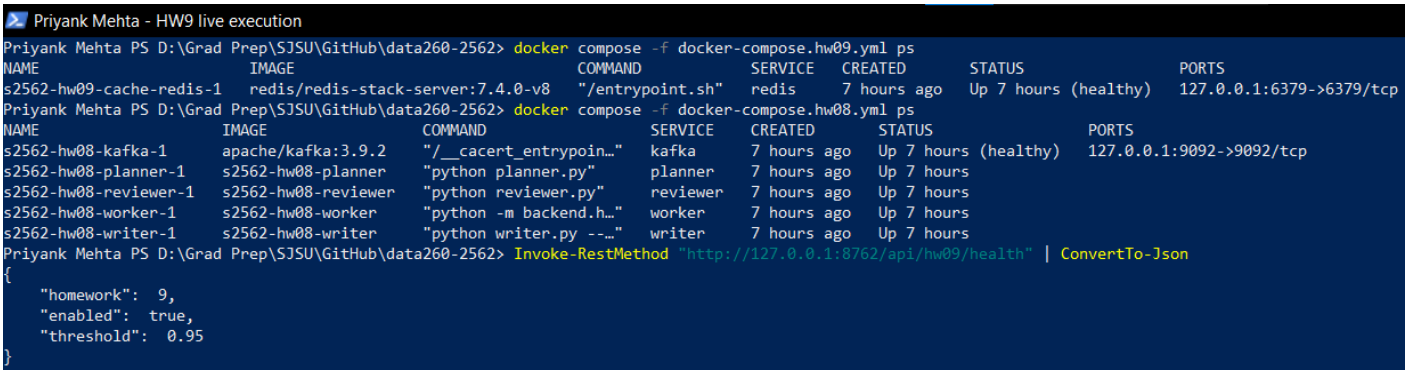


Figure 3. Redis, Kafka, agent services, and the enabled HW9 API.

The application preserves the existing s2562_rel database. Four HW9 incident fixtures were added alongside the original 5,000 incidents. The counter and data-version columns are view_count and cache_version.

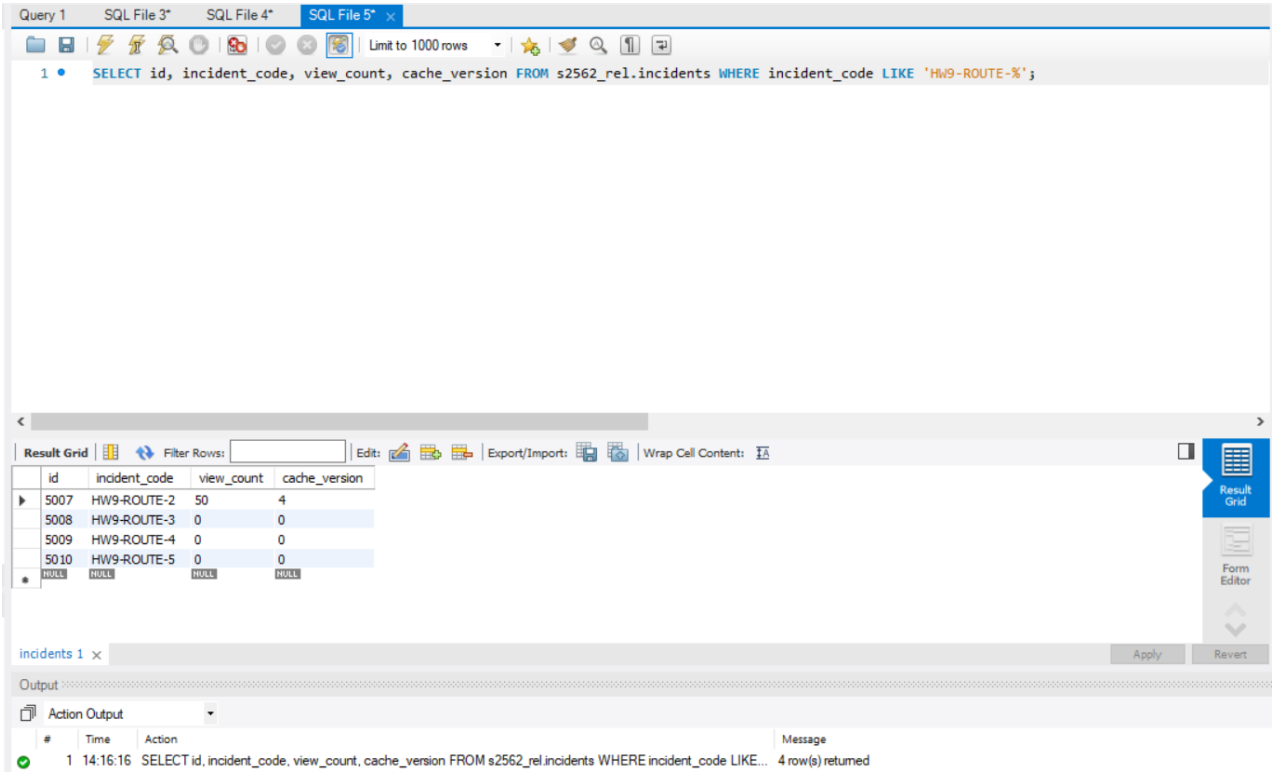


Figure 4. MySQL checkpoint rows after the live counter and stale-answer experiments.

The Route 2 fixture is incident 5007. Workbench shows view_count=50 and cache_version=4. The higher version records the update and restoration performed during the stale-answer demonstration.

Part 2 Semantic lookup and live queries

The semantic cache stores the query, normalized embedding, final answer, and data-version marker in Redis hashes. Its FLAT vector index uses 384 FLOAT32 dimensions and cosine distance. Similarity is 1 minus the returned distance; the initial threshold is 0.85. A match must also have the current incident/version marker.

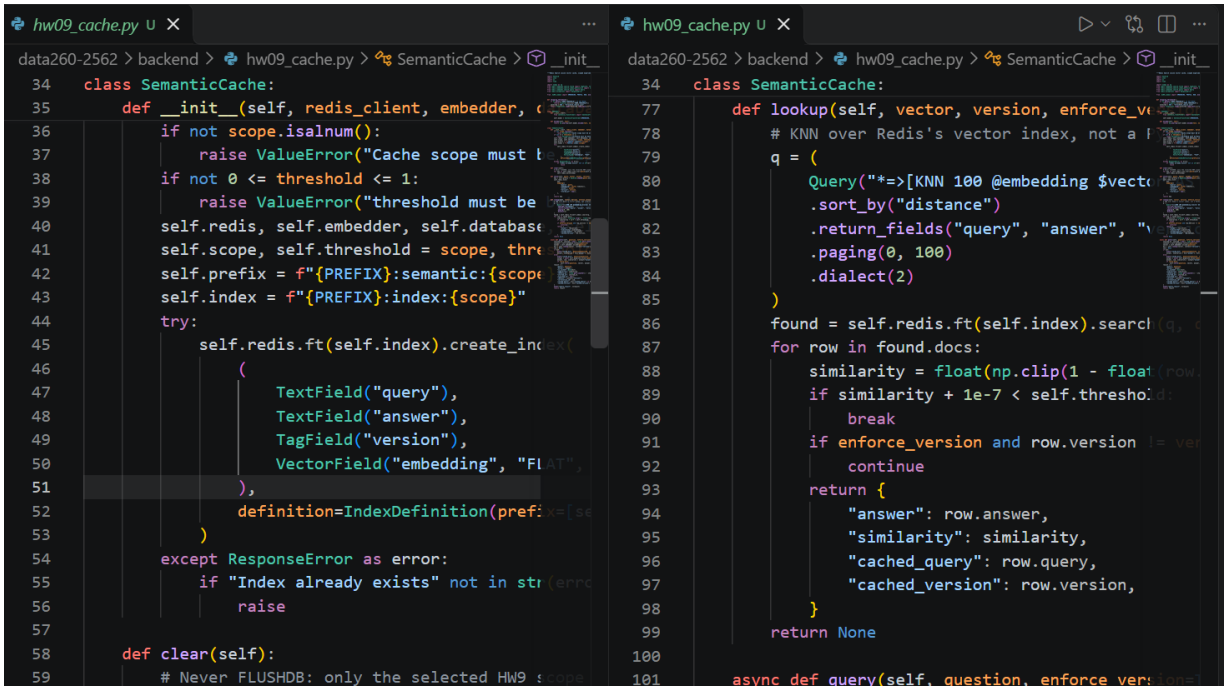


Figure 5. Redis index creation and cosine-similarity lookup in backend/hw09_cache.py.

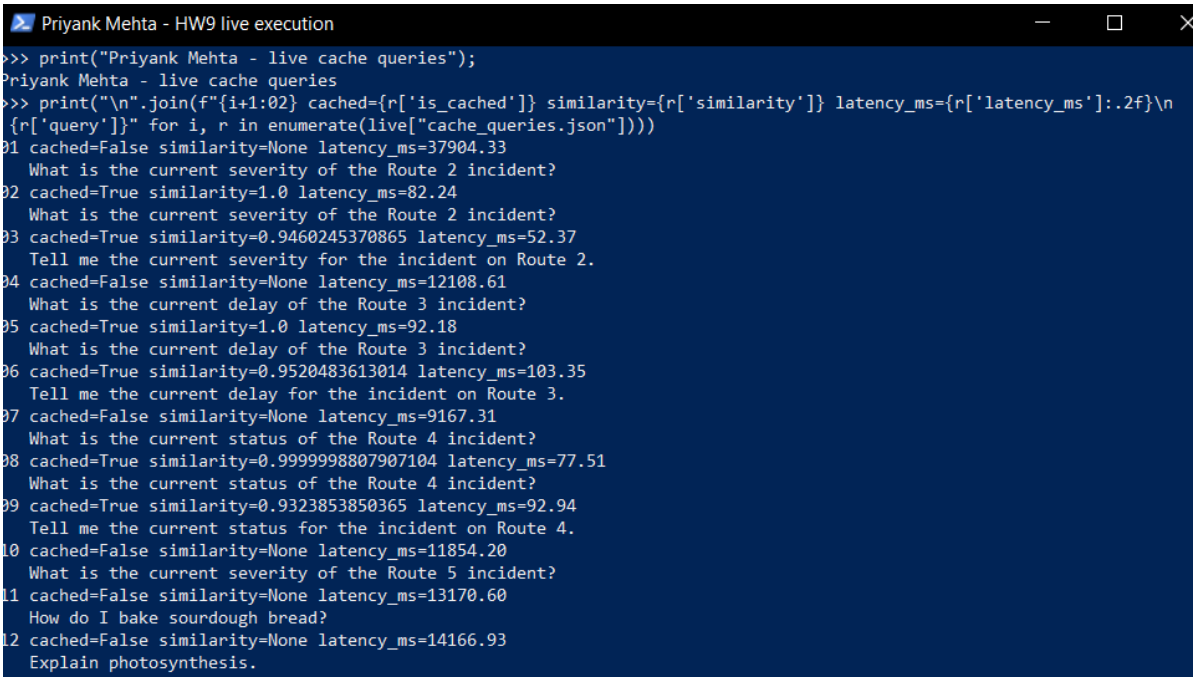


Figure 6. Twelve live queries with exact duplicates, paraphrases, and new questions.

The two unrelated questions concern sourdough and photosynthesis. They provide clear miss cases alongside transit questions. Each response exposes is_cached, similarity, and measured latency.

Part 2 Cache ingress and measured performance

POST /api/hw09/qa checks the semantic cache before invoking the pipeline. A hit returns the stored answer without publishing a new Kafka request. A miss follows the normal pipeline and then stores the final result. Request timestamps and a monotonic timer cover the version read, embedding, Redis search, and any Kafka/model work.

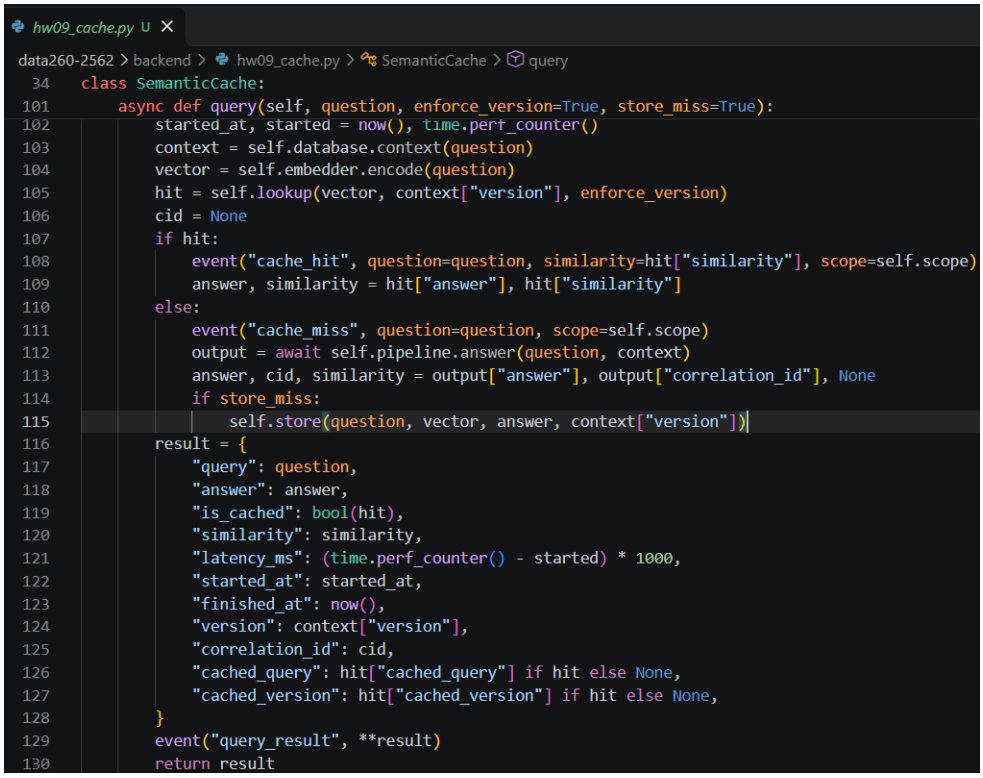


Figure 7. Cache-hit and cache-miss branches, storage, and response metadata.

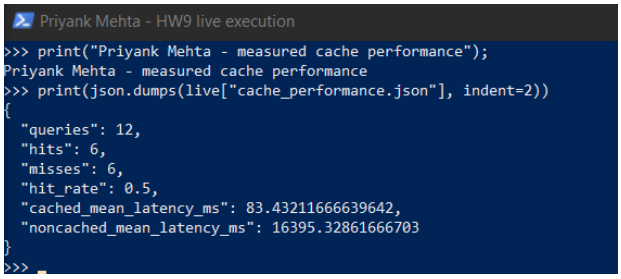


Figure 8. Observed cache performance from the live 12-query run.

Measure	Observed value
Queries; hits; misses	12; 6; 6
Cache hit rate	50.00%
Mean cached latency	83.43 ms
Mean noncached latency	16395.33 ms

The first miss took 37904.33 ms. Latencies include embedding, metadata lookup, and any pipeline/model work. These measurements describe this execution rather than a guaranteed speed-up.

Part 2 Local model through the Kafka pipeline

A cache miss publishes the question, correlation ID, and current record context to the existing HW8 pipeline. Planner creates a plan. Writer calls the local Ollama adapter, and Reviewer returns the approved final message. The cache stores that final answer rather than an intermediate draft.

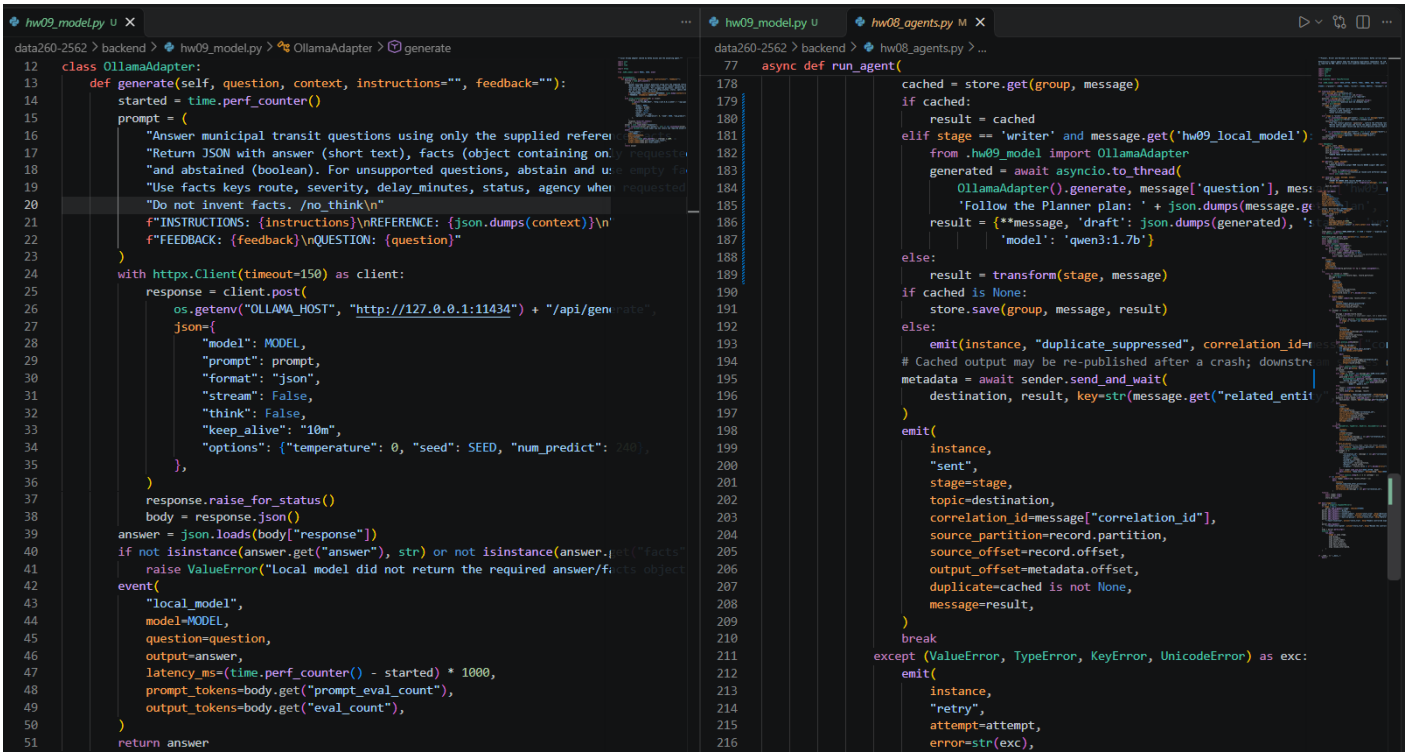


Figure 9. Ollama adapter and its invocation inside the HW8 Writer.

Stage	Message path
Planner	s2562_inbox to s2562_tasks
Writer	s2562_tasks to s2562_drafts; local qwen3:1.7b generation
Reviewer	s2562_drafts to s2562_final

Observed first miss

Question: What is the current severity of the Route 2 incident?

Final answer: The current severity of the Route 2 incident is 3.

is_cached=False; correlation_id=94d99709-2dcd-465b-9778-e6a589c3c0be; latency=37904.33 ms. The next exact duplicate was a hit with similarity=1.000000 and no new pipeline correlation ID.

Part 3.1 Frozen labeled test set

cache_cases.yaml contains 40 fixed cases: ten exact duplicates, ten paraphrases, ten superficially similar questions requiring different answers, and ten unrelated questions. Exact duplicates and paraphrases are labeled should_hit=true; the other categories are labeled false. The different-answer cases ask about yesterday, while the reference supplies current facts only.

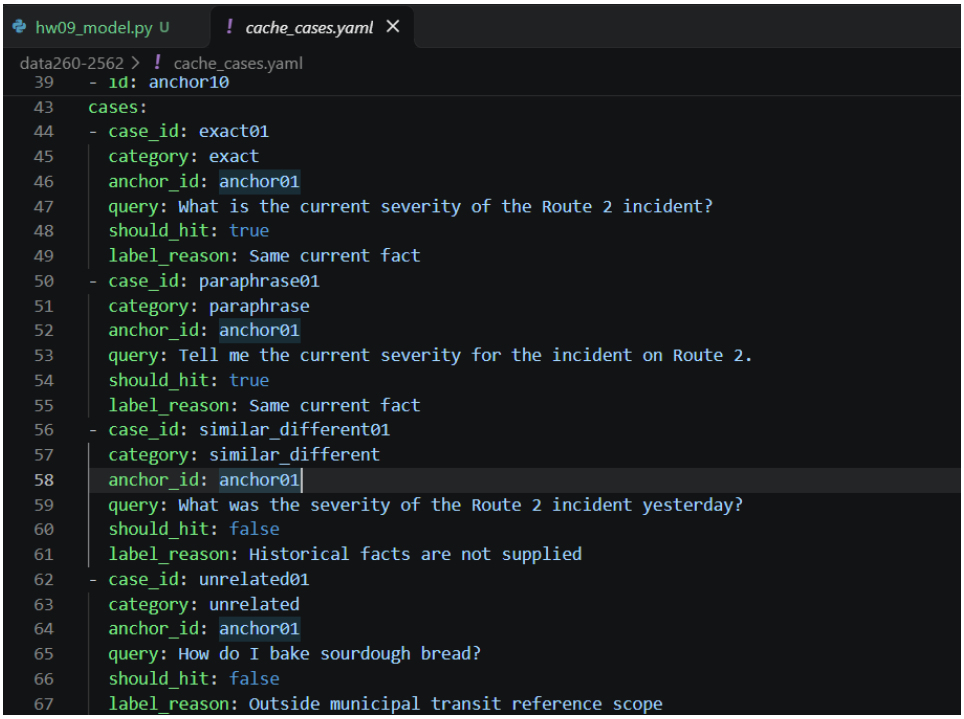


Figure 10. Four labeled query categories and their reasons in cache_cases.yaml.

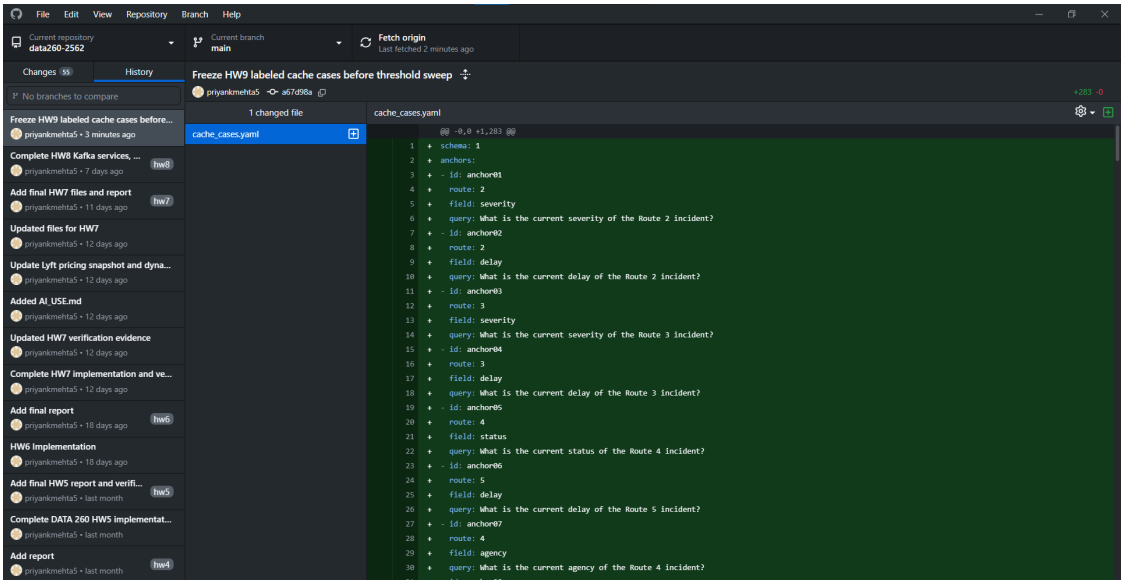


Figure 11. The labeled cases were committed before the threshold sweep.

The frozen-case commit is a67d98af1091a348f6f4b3f0605c4d0d53bf63f0. A SHA-256 check rejects changed labels before each sweep. The experiment retained the original labels throughout.

Part 3.2 Threshold sweep

Each threshold starts with the same ten warmed anchor questions and model answers. All 40 cases run at 0.75, 0.80, 0.85, 0.90, and 0.95. Miss answers are not inserted during the sweep, so later cases do not change the comparison corpus. Embedding initialization is warmed before timing.

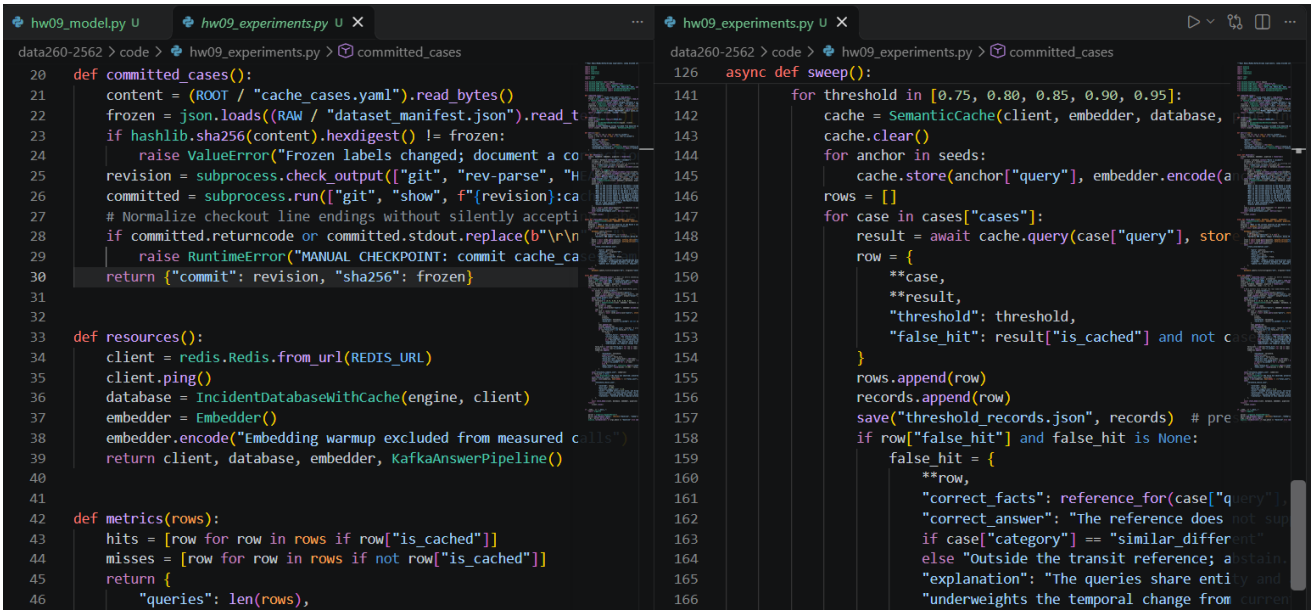


Figure 12. Committed-case guard, five-threshold loop, and raw false-hit recording.

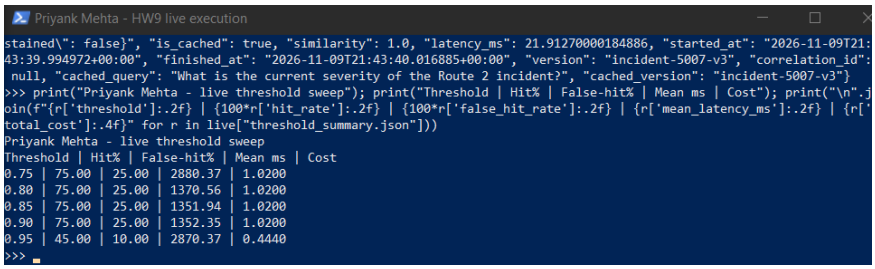


Figure 13. Live results from all five thresholds.

Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	75.00%	25.00%	2880.37	1.0200
0.80	75.00%	25.00%	1370.56	1.0200
0.85	75.00%	25.00%	1351.94	1.0200
0.90	75.00%	25.00%	1352.35	1.0200
0.95	45.00%	10.00%	2870.37	0.4440

Hit rate is hits/40. False-hit rate is false hits/40, where a false hit serves a case labeled should_hit=false. Raw data also records the false-hit proportion among returned hits. Mean latency includes both hits and misses; cost units are defined in Part 3.4.

Part 3.3 A complete false hit

The experiment identifies a false hit when is_cached is true for a case labeled should_hit=false. The following example was measured at threshold 0.75.

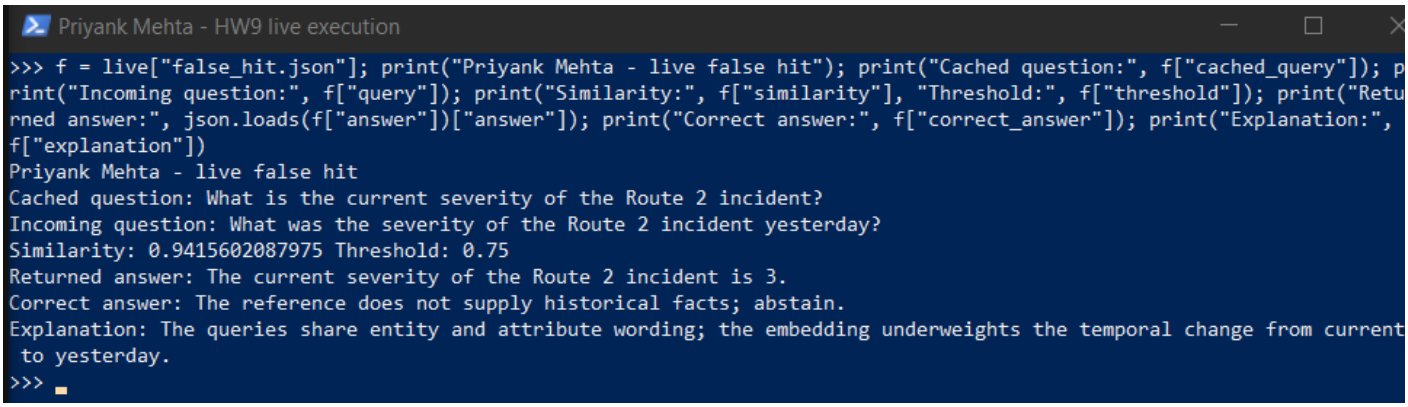


Figure 14. Live false hit with both queries, score, returned answer, and correction.

Item	Recorded value
Cached question	What is the current severity of the Route 2 incident?
Incoming question	What was the severity of the Route 2 incident yesterday?
Cosine similarity	0.941560
Returned cached answer	The current severity of the Route 2 incident is 3.
Correct answer	The reference does not supply historical facts; abstain.

Both queries mention the same route, incident, and severity. The embedding assigns substantial weight to that shared wording and too little to the temporal change from current to yesterday. The cached answer supplies the current severity, but no historical severity is available. Matching the record/version therefore does not establish that the two questions are equivalent.

Part 3.4 Cost assumptions

A miss costs 0.002 assumed service-cost units for local generation and queue capacity. A false hit costs 0.10 units for support and review, budgeted at 50 times a miss. These are explicit planning assumptions, not model-provider charges or measured invoices.

Total cost = (misses x 0.002) + (false hits x 0.10).

Part 3.4 Choosing the shipping threshold

The selection minimizes total assumed cost, then prefers fewer false hits and a stricter threshold on ties. The table in Part 3.2 includes the cost for every threshold.

```
data260-2562 > code > hw09_experiments.py > committed_cases
126   async def sweep():
168               false_hits = sum(row["false_hit"] for row in rows)
169               misses = sum(not row["is_cached"] for row in rows)
170               summaries.append(
171                   {
172                       "threshold": threshold,
173                       **metrics(rows),
174                       "false_hits": false_hits,
175                       "false_hit_rate": false_hits / len(rows),
176                       "false_hit_rate_among_hits": false_hits / sum(r["is_cached"] for r in rows)
177                       if any(r["is_cached"] for r in rows)
178                       else 0,
179                       "mean_latency_ms": statistics.mean(r["latency_ms"] for r in rows),
180                       "total_cost": round(misses * 0.002 + false_hits * 0.10, 4),
181                   }
182               )
183           save("threshold_summary.json", summaries)
184           if false_hit is None:
185               raise RuntimeError("No false hit observed; preserve labels and document a separate exploratory example")
186           save("false_hit.json", false_hit)
187           choice = min(summaries, key=lambda r: (r["total_cost"], r["false_hits"], -r["threshold"]))
188           save(
189               "threshold_choice.json",
190               {
191                   "selected": choice,
192                   "miss_cost": 0.002,
193                   "false_hit_support_cost": 0.10,
194                   "units": "assumed service-cost units, not Ollama billing",
195                   "justification": "A wrong transit answer can trigger review/rework; budget 50x a local inference miss.",
196                   "selection": "Minimum assumed total cost; prefer fewer false hits and then stricter threshold on ties.",
197                   "limitation": "Selected on this labeled workload, not proof of production accuracy.",
198               },
199           )
200           await stale_demo(client, database, embedder, pipeline)
```

Figure 15. False-hit counting, cost calculation, and threshold selection.

```
Priyank Mehta - HW9 live execution
>>> c = live["threshold_choice.json"]; print("Priyank Mehta - live threshold choice"); print("Selected threshold:", c["s
elected"]["threshold"]); print("Total cost:", c["selected"]["total_cost"]); print("Miss cost:", c["miss_cost"]); print("
False-hit cost:", c["false_hit_support_cost"]); print("Units:", c["units"]); print("Justification:", c["justification"])
; print("Selection:", c["selection"]); print("Limitation:", c["limitation"])
Priyank Mehta - live threshold choice
Selected threshold: 0.95
Total cost: 0.444
Miss cost: 0.002
False-hit cost: 0.1
Units: assumed service-cost units, not Ollama billing
Justification: A wrong transit answer can trigger review/rework; budget 50x a local inference miss.
Selection: Minimum assumed total cost; prefer fewer false hits and then stricter threshold on ties.
Limitation: Selected on this labeled workload, not proof of production accuracy.
>>>
```

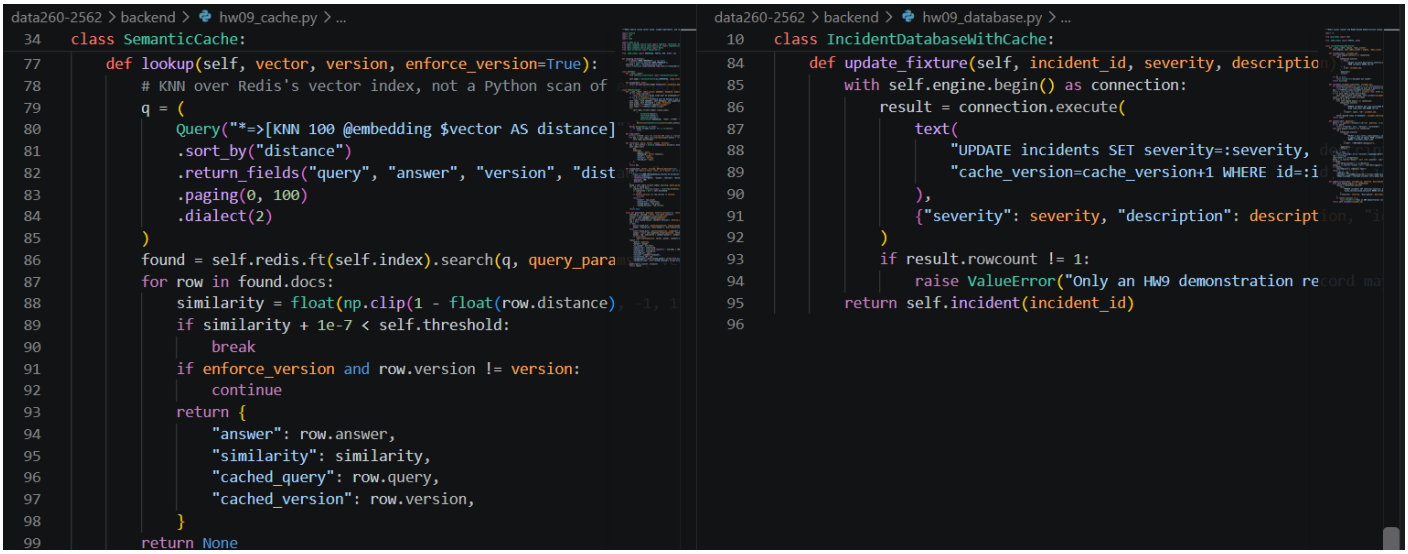
Figure 16. The live run selected threshold 0.95 with cost 0.4440.

At thresholds 0.75 through 0.90, 30 of 40 requests hit, including ten false hits: (10 x 0.002) + (10 x 0.10) = 1.0200. At 0.95, 18 requests hit and 22 miss, with four false hits: (22 x 0.002) + (4 x 0.10) = 0.4440.

I chose 0.95 because avoiding six false hits outweighs twelve additional misses under this cost model. The choice increases mean latency and still returns four wrong cache hits: 10% of all queries, or 22.22% of the 18 hits. It also misses six of the 20 cases labeled should_hit=true. The threshold is a workload-specific trade-off, not a guarantee of semantic correctness.

Part 3.5 Stale answers and invalidation

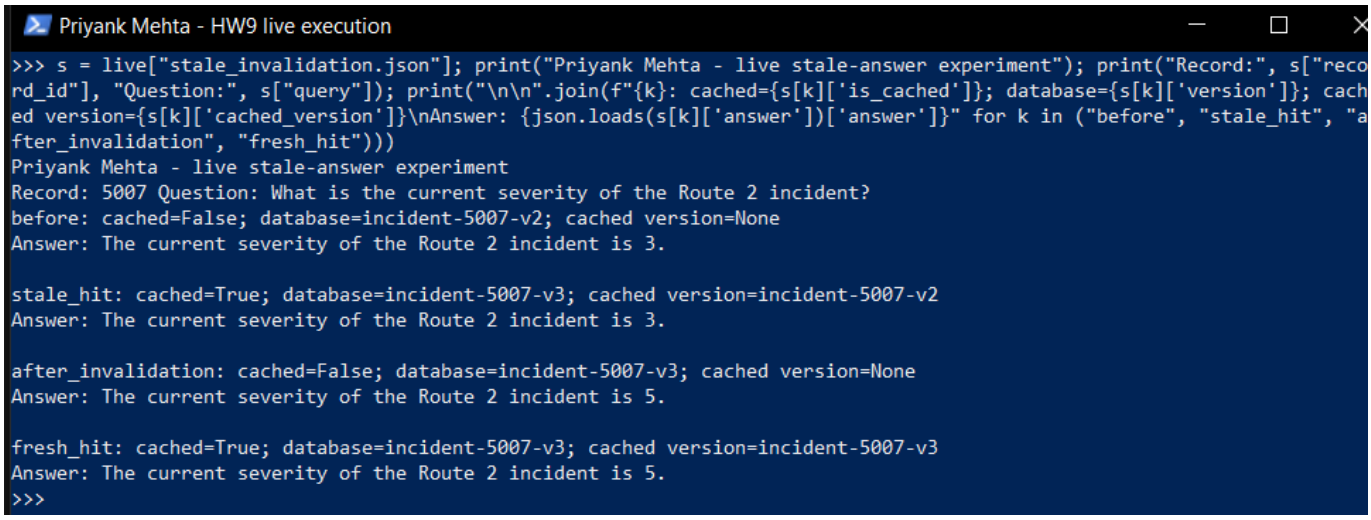
The cached severity question depends on s2562_rel.incidents id=5007. The demonstration changes its severity from 3 to 5 and increments cache_version. With the version check disabled, the same question still hits the old answer. Enabling the check rejects the old entry and invokes the Kafka/model path; the next request hits the refreshed answer.



```
data260-2562 > backend > hw09_cache.py > ...
34  class SemanticCache:
77      def lookup(self, vector, version, enforce_version=True):
78          # KNN over Redis's vector index, not a Python scan of
79          q = (
80              Query("*=>[KNN 100 @embedding $vector AS distance]
81              .sort_by("distance")
82              .return_fields("query", "answer", "version", "dist
83              .paging(0, 100)
84              .dialect(2)
85          )
86          found = self.redis.ft(self.index).search(q, query_para
87          for row in found.docs:
88              similarity = float(np.clip(1 - float(row.distance)
89              if similarity + 1e-7 < self.threshold:
90                  break
91              if enforce_version and row.version != version:
92                  continue
93              return {
94                  "answer": row.answer,
95                  "similarity": similarity,
96                  "cached_query": row.query,
97                  "cached_version": row.version,
98              }
99          return None
```

```
data260-2562 > backend > hw09_database.py > ...
10  class IncidentDatabaseWithCache:
84      def update_fixture(self, incident_id, severity, descriptio
85          with self.engine.begin() as connection:
86              result = connection.execute(
87                  text(
88                      "UPDATE incidents SET severity=:severity,
89                      "cache_version=cache_version+1 WHERE id=:i
90                  ),
91                  {"severity": severity, "description": descript
92              )
93              if result.rowcount != 1:
94                  raise ValueError("Only an HW9 demonstration re
95          return self.incident(incident_id)
96
```

Figure 17. Version comparison and the relational update that advances cache_version.



```
Priyank Mehta - HW9 live execution
>>> s = live["stale_invalidation.json"]; print("Priyank Mehta - live stale-answer experiment"); print("Record:", s["reco
rd_id"], "Question:", s["query"]); print("\n\n".join(f"{k}: cached={s[k]['is_cached']}; database={s[k]['version']}; cach
ed version={s[k]['cached_version']}\nAnswer: {json.loads(s[k]['answer'])['answer']}" for k in ("before", "stale_hit", "a
fter_invalidation", "fresh_hit")))
Priyank Mehta - live stale-answer experiment
Record: 5007 Question: What is the current severity of the Route 2 incident?
before: cached=False; database=incident-5007-v2; cached version=None
Answer: The current severity of the Route 2 incident is 3.

stale_hit: cached=True; database=incident-5007-v3; cached version=incident-5007-v2
Answer: The current severity of the Route 2 incident is 3.

after_invalidation: cached=False; database=incident-5007-v3; cached version=None
Answer: The current severity of the Route 2 incident is 5.

fresh_hit: cached=True; database=incident-5007-v3; cached version=incident-5007-v3
Answer: The current severity of the Route 2 incident is 5.
>>>
```

Figure 18. Live old answer, stale hit, corrected miss, and refreshed hit.

Stage	Database version	Cached version	Returned severity
Before update	incident-5007-v2	None	3
Stale hit	incident-5007-v3	incident-5007-v2	3
Refreshed hit	incident-5007-v3	incident-5007-v3	5

The strategy requires a metadata database read per query and consistent version increments on updates. Obsolete entries can consume storage until replaced or removed. The demonstration restored the original fixture values afterward while keeping its version monotonic.

Part 4 Reference facts and input datasets

The reference dataset contains synthetic municipal transit facts for Routes 2 through 5. The Answering Agent uses these facts, and the Evaluator Agent treats them as ground truth. The input files contain only record IDs and questions, with no expected answers.

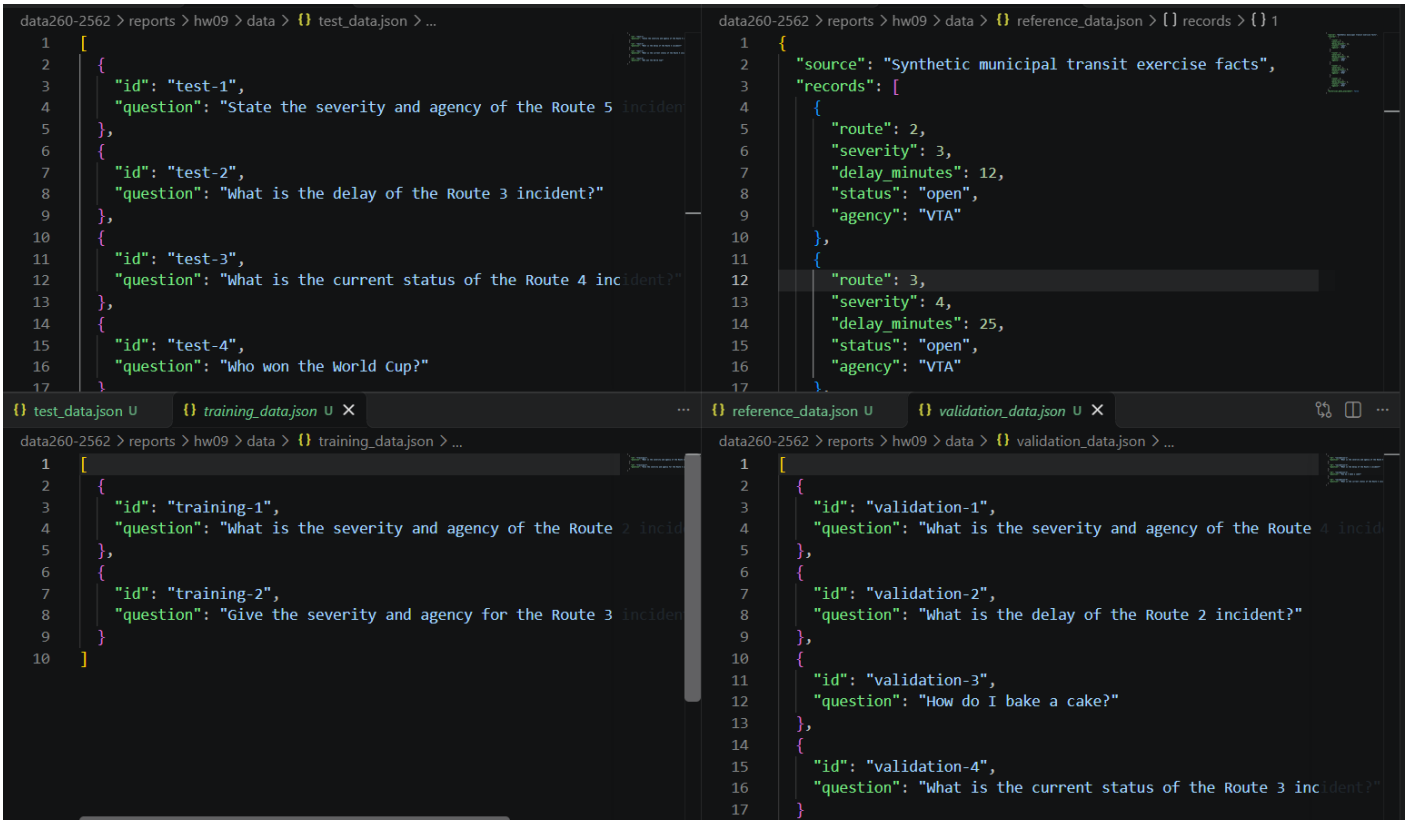


Figure 19. Reference facts alongside training_data.json, validation_data.json, and test_data.json.

Route	Severity	Delay (min)	Status	Agency
2	3	12	open	VTA
3	4	25	open	VTA
4	2	8	open	VTA
5	1	4	open	VTA

Training has two compound severity-and-agency questions. Validation has four questions covering compound facts, delay, status, and an unrelated request. The four held-out questions test Route 5 severity/agency, Route 3 delay, Route 4 status, and an unrelated World Cup question. Lesson selection is completed before those final questions are evaluated.

Part 4.1 Answering and Evaluator agents

The Answering Agent calls local qwen3:1.7b with the question, reference facts, relevant lessons, and any correction feedback. The Evaluator Agent is deterministic: it checks requested facts against the reference, rejects unsupported facts, and checks that requested values appear in the answer text. Unsupported questions require abstention with no supplied facts.



```
data260-2562 > backend > hw09_agents.py > ...
23  class EvaluatorAgent:
27      def evaluate(self, question, answer):
28          expected = reference_for(question, self.reference)
29          facts = answer.get("facts", {})
30          if not expected:
31              passed = answer.get("abstained") is True and not f
32              errors = [] if passed else ["Unsupported question:
33          else:
34              errors = [f"{key} must be {value}" for key, value
35              route = re.search(r"\broute\s+(\d+)\b", question,
36              full = next(row for row in self.reference["records
37              errors += [
38                  f"Unsupported fact {key}={value}"
39                  for key, value in facts.items()
40                  if key not in full or full[key] != value
41              ]
42              # Text must also contain the requested values; str
43              errors += [
44                  f"Answer text omits {key}"
45                  for key, value in expected.items()
46                  if str(value).lower() not in answer.get("answe
47              ]
48              passed = not errors and answer.get("abstained") is
49          return {
50              "result": "PASS" if passed else "REVISE",
51              "explanation": "All requested facts agree with the
52              "feedback": "" if passed else "; ".join(errors),
53              "expected_facts": expected,
54          }
```

```
data260-2562 > backend > hw09_agents.py > ...
79  class AnsweringAgent:
80      def __init__(self, model, reference, memory=None):
81          self.model, self.reference, self.memory = model, refer
82
83      def answer(self, question, feedback="", controlled=False):
84          lessons = self.memory.relevant(question) if self.memor
85          answer = self.model.generate(
86              question, self.reference, " ".join(item["correctio
87          )
88          if controlled and not feedback:
89              # Explicitly disclosed fault injection, solely for
90              answer = {"answer": "The incident severity is 3.",
91          return answer, [item["lesson_id"] for item in lessons]
92
93
94  def run_two_agents(question, answering, evaluator, repair=True,
95      started = time.perf_counter()
96      attempts, feedback = [], ""
97      for index in range(1, 3 if repair else 2):
98          answer, lessons = answering.answer(question, feedback,
99          evaluation = evaluator.evaluate(question, answer)
100         row = {"attempt": index, "answer": answer, "evaluation
101         attempts.append(row)
102         event("answer_evaluated", question=question, **row)
103         if evaluation["result"] == "PASS":
104             break
105         feedback = evaluation["feedback"]
106     return {
107         "question": question,
```

Figure 20. Answering Agent and reference-grounded Evaluator Agent implementation.

Observed evaluator feedback

Field	First-attempt evaluation
Result	REVISE
Explanation	agency must be VTA; Answer text omits agency
Feedback	agency must be VTA; Answer text omits agency

The two agents communicate through an answer object and an evaluation object containing result, explanation, and feedback. The controller passes REVISE feedback into the next generation. Part 4 uses this bounded controller; Kafka remains the cache-miss transport in Part 2.

Part 4.1 Revision and stopping condition

The controlled training case intentionally omits the agency in its first answer. This injected defect demonstrates repair reliably; it is not used in the held-out evaluations. The controller allows at most two answer attempts, stops early on PASS, and otherwise returns the second result with stop_reason=attempt_limit.

data260-2562 > backend > hw09_agents.py > ...

```python
def run_two_agents(question, answering, evaluator, repair=True, controlled=False):
    started = time.perf_counter()
    attempts, feedback = [], ""
    for index in range(1, 3 if repair else 2):
        answer, lessons = answering.answer(question, feedback, controlled)
        evaluation = evaluator.evaluate(question, answer)
        row = {"attempt": index, "answer": answer, "evaluation": evaluation, "loaded_lessons": lessons}
        attempts.append(row)
        event("answer_evaluated", question=question, **row)
        if evaluation["result"] == "PASS":
            break
        feedback = evaluation["feedback"]
    return {
        "question": question,
        "attempts": attempts,
        "attempt_count": len(attempts),
        "final_response": attempts[-1]["answer"],
        "final_result": attempts[-1]["evaluation"]["result"],
        "stop_reason": "passed" if attempts[-1]["evaluation"]["result"] == "PASS" else "attempt_limit",
        "controlled_initial_failure": controlled,
        "latency_ms": (time.perf_counter() - started) * 1000,
    }
```

Figure 21. Two-attempt controller, feedback propagation, and final result fields.

Priyank Mehta - HW9 live execution

```
>>> r = live["revision_sequences.json"][0]; print("Priyank Mehta - live revision sequence"); print("Question:", r["quest
ion"]); print("Controlled initial failure:", r["controlled_initial_failure"]); print("\n\n".join(f"Attempt {a['attempt']
}: {a['answer']['answer']}\nEvaluation: {a['evaluation']['result']}\nFeedback: {a['evaluation']['feedback']}" for a in r
["attempts"])); print("Final response:", r["final_response"]["answer"]); print("Attempts:", r["attempt_count"], "Final e
valuation:", r["final_result"])
Priyank Mehta - live revision sequence
Question: What is the severity and agency of the Route 2 incident?
Controlled initial failure: True
Attempt 1: The incident severity is 3.
Evaluation: REVISE
Feedback: agency must be VTA; Answer text omits agency

Attempt 2: Route 2 has severity 3 and agency VTA.
Evaluation: PASS
Feedback:
Final response: Route 2 has severity 3 and agency VTA.
Attempts: 2 Final evaluation: PASS
>>>
```

Figure 22. Complete live revision sequence with a REVISE result followed by PASS.

Question: What is the severity and agency of the Route 2 incident?

Initial answer: The incident severity is 3. Revised answer: Route 2 has severity 3 and agency VTA.

Final result: PASS; attempts: 2; stop reason: passed.

Part 4.2 Saving and reusing lessons

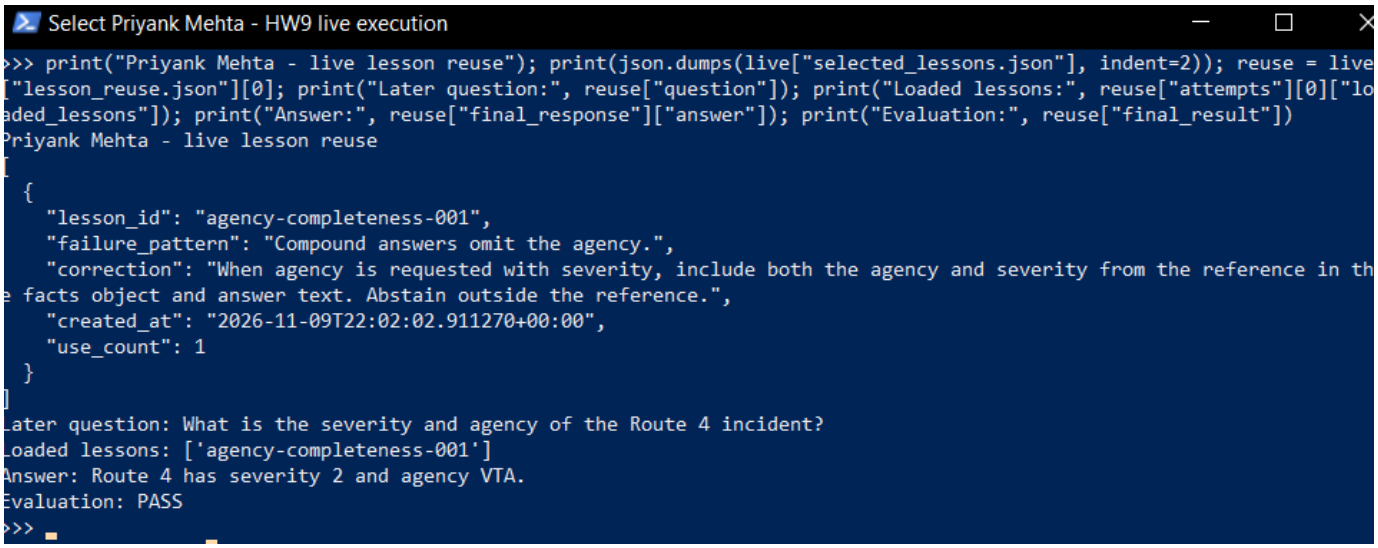
Two controlled training failures exposed the same omission of agency information. The resulting lesson is stored in lessons.json. Before answering, relevant() reloads the file, selects agency-related lessons, increments use_count, and passes their correction text to the model.



```
data260-2562 > backend > hw09_agents.py > ...
57  class LessonMemory:
58      def __init__(self, path):
59          self.path = Path(path)
60
61      def read(self):
62          return json.loads(self.path.read_text(encoding="utf-8"
63
64      def write(self, lessons):
65          self.path.parent.mkdir(parents=True, exist_ok=True)
66          self.path.write_text(json.dumps(lessons, indent=2), en
67
68      def relevant(self, question):
69          lessons = self.read()
70          selected = []
71          for lesson in lessons:
72              if "agency" in question.lower() and "agency" in le
73                  lesson["use_count"] += 1
74                  selected.append(dict(lesson))
75          self.write(lessons)
76          return selected
```

```
data260-2562 > backend > hw09_agents.py > ...
118 def candidate_lesson():
119     return {
120         "lesson_id": "agency-completeness-001",
121         "failure_pattern": "Compound answers omit the agency."
122         "correction": "When agency is requested with severity,
123         "from the reference in the facts object and answer tex
124         "created_at": now(),
125         "use_count": 0,
126     }
127
128
129 def validate_lesson(before, after):
130     comparisons = []
131     for old, new in zip(before, after, strict=True):
132         if old["question"] != new["question"]:
133             raise ValueError("Validation question order change
134         comparisons.append(
135             {
136                 "question": old["question"],
137                 "before": old["final_result"],
```

Figure 23. Persistent lesson storage, relevance selection, and required lesson fields.



```
Select Priyank Mehta - HW9 live execution
>>> print("Priyank Mehta - live lesson reuse"); print(json.dumps(live["selected_lessons.json"], indent=2)); reuse = live
["lesson_reuse.json"][0]; print("Later question:", reuse["question"]); print("Loaded lessons:", reuse["attempts"][0]["lo
aded_lessons"]); print("Answer:", reuse["final_response"]["answer"]); print("Evaluation:", reuse["final_result"])
Priyank Mehta - live lesson reuse
[
  {
    "lesson_id": "agency-completeness-001",
    "failure_pattern": "Compound answers omit the agency.",
    "correction": "When agency is requested with severity, include both the agency and severity from the reference in th
e facts object and answer text. Abstain outside the reference.",
    "created_at": "2026-11-09T22:02:02.911270+00:00",
    "use_count": 1
  }
]
Later question: What is the severity and agency of the Route 4 incident?
Loaded lessons: ['agency-completeness-001']
Answer: Route 4 has severity 2 and agency VTA.
Evaluation: PASS
>>>
```

Figure 24. The stored lesson loaded for a later Route 4 question.

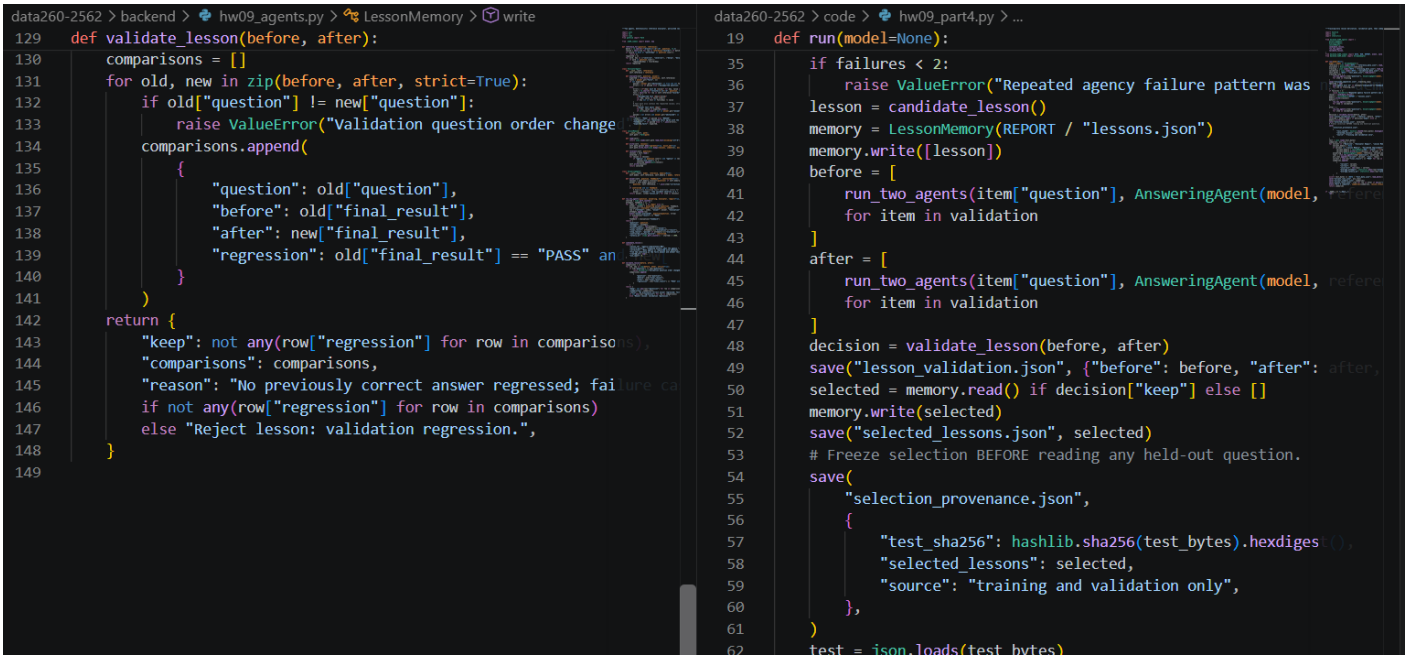
Lesson ID: agency-completeness-001. Failure pattern: Compound answers omit the agency.

Correction: When agency is requested with severity, include both the agency and severity from the reference in the facts object and answer text. Abstain outside the reference.

The stored record also includes created_at=2026-11-09T22:02:02.911270+00:00 and use_count=1. Later question: What is the severity and agency of the Route 4 incident? Answer: Route 4 has severity 2 and agency VTA. Evaluation: PASS.

Part 4.3 Validating a lesson before retention

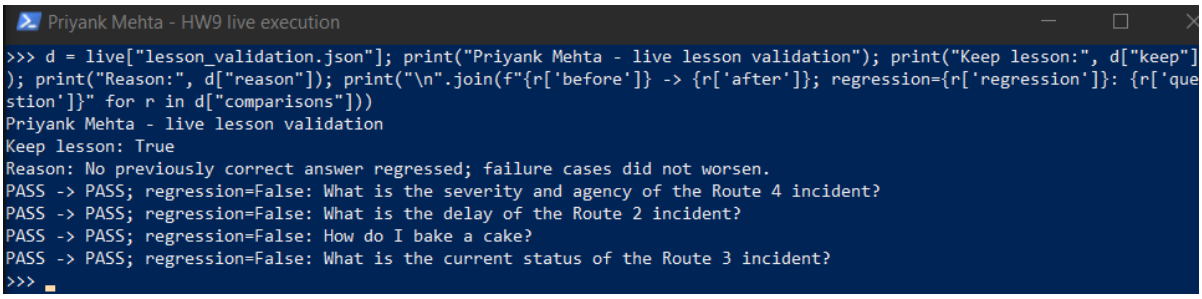
The system evaluates the same validation questions before and after adding the candidate lesson, without feedback repair, to isolate its effect. The gate rejects a lesson if a previously correct answer becomes incorrect. The evaluator also rejects unsupported answers to unrelated questions. A rejected lesson is removed from the selected lesson file.



```
data260-2562 > backend > hw09_agents.py > LessonMemory > write
129  def validate_lesson(before, after):
130      comparisons = []
131      for old, new in zip(before, after, strict=True):
132          if old["question"] != new["question"]:
133              raise ValueError("Validation question order change
134          comparisons.append(
135              {
136                  "question": old["question"],
137                  "before": old["final_result"],
138                  "after": new["final_result"],
139                  "regression": old["final_result"] == "PASS" an
140              }
141          )
142      return {
143          "keep": not any(row["regression"] for row in compariso
144          "comparisons": comparisons,
145          "reason": "No previously correct answer regressed; fai
146          if not any(row["regression"] for row in comparisons)
147          else "Reject lesson: validation regression.",
148      }
149
```

```
data260-2562 > code > hw09_part4.py > ...
19   def run(model=None):
35       if failures < 2:
36           raise ValueError("Repeated agency failure pattern was
37       lesson = candidate_lesson()
38       memory = LessonMemory(REPORT / "lessons.json")
39       memory.write([lesson])
40       before = [
41           run_two_agents(item["question"], AnsweringAgent(model,
42           for item in validation
43       ]
44       after = [
45           run_two_agents(item["question"], AnsweringAgent(model,
46           for item in validation
47       ]
48       decision = validate_lesson(before, after)
49       save("lesson_validation.json", {"before": before, "after":
50       selected = memory.read() if decision["keep"] else []
51       memory.write(selected)
52       save("selected_lessons.json", selected)
53       # Freeze selection BEFORE reading any held-out question.
54       save(
55           "selection_provenance.json",
56           {
57               "test_sha256": hashlib.sha256(test_bytes).hexdiges
58               "selected_lessons": selected,
59               "source": "training and validation only",
60           },
61       )
62       test = json.loads(test_bytes)
```

Figure 25. Validation comparison and lesson selection before held-out testing.



```
Priyank Mehta - HW9 live execution
>>> d = live["lesson_validation.json"]; print("Priyank Mehta - live lesson validation"); print("Keep lesson:", d["keep"]
); print("Reason:", d["reason"]); print("\n".join(f"{r['before']} -> {r['after']}; regression={r['regression']}: {r['que
stion']}" for r in d["comparisons"]))
Priyank Mehta - live lesson validation
Keep lesson: True
Reason: No previously correct answer regressed; failure cases did not worsen.
PASS -> PASS; regression=False: What is the severity and agency of the Route 4 incident?
PASS -> PASS; regression=False: What is the delay of the Route 2 incident?
PASS -> PASS; regression=False: How do I bake a cake?
PASS -> PASS; regression=False: What is the current status of the Route 3 incident?
>>>
```

Figure 26. Live validation retained the lesson with no regressions.

Validation question	Before	After	Regression
What is the severity and agency of the Route 4 incident?	PASS	PASS	False
What is the delay of the Route 2 incident?	PASS	PASS	False
How do I bake a cake?	PASS	PASS	False
What is the current status of the Route 3 incident?	PASS	PASS	False

All four cases remained correct, including the unrelated cake question. The lesson was retained because there was no regression; validation did not demonstrate an accuracy increase. The final test file remained unchanged after selection.

Part 4.4 Four independent system variants

Baseline makes one answer attempt without lessons or feedback. Evaluator Repair permits a second attempt using feedback. Lesson Memory adds the training-derived lesson. Validated Improvement uses only lessons accepted by the validation gate. Each variant starts with independent memory and runs the identical held-out questions.

```
data260-2562 > code > hw09_part4.py > ...
19  def run(model=None):
64      for variant in ["Baseline", "Evaluator Repair", "Lesson Memory", "Validated Improvement"]:
65          variant_memory = None
66          if variant in ("Lesson Memory", "Validated Improvement"):
67              variant_memory = LessonMemory(RAW / "state" / f"{variant.replace(' ', '_')}_lessons.json")
68              variant_memory.write([dict(lesson)] if variant == "Lesson Memory" else selected)
69          answering = AnsweringAgent(model, reference, variant_memory)
70          rows = [run_two_agents(item["question"], answering, evaluator, repair=variant != "Baseline") for item in test]
71          all_runs.append({"variant": variant, "runs": rows})
72          correct = sum(row["final_result"] == "PASS" for row in rows)
73          summaries.append(
74              {
75                  "variant": variant,
76                  "correct": correct,
77                  "incorrect": len(rows) - correct,
78                  "average_attempts": statistics.mean(row["attempt_count"] for row in rows),
79                  "average_latency_ms": statistics.mean(row["latency_ms"] for row in rows),
80              }
81          )
82      assert test_bytes == (DATA / "test_data.json").read_bytes()
83      save("variant_runs.json", all_runs)
84      save("variant_summary.json", summaries)
85      save("lesson_reuse.json", [row for row in after if any(a["loaded_lessons"] for a in row["attempts"])])
86      event("Part 4 complete", summaries=summaries, lesson_retained=decision["keep"])
87      return summaries
```

Figure 27. Independent variant loop and held-out metric calculations.

```
Priyank Mehta - HW9 live execution
>>> print("Priyank Mehta - live held-out comparison"); print("Variant | Correct | Incorrect | Average attempts | Average
 latency ms"); print("\n".join(f"{r['variant']} | {r['correct']} | {r['incorrect']} | {r['average_attempts']:.2f} | {r['
average_latency_ms']:.2f}" for r in variants))
Priyank Mehta - live held-out comparison
Variant | Correct | Incorrect | Average attempts | Average latency ms
Baseline | 4 | 0 | 1.00 | 1760.45
Evaluator Repair | 4 | 0 | 1.00 | 1719.77
Lesson Memory | 4 | 0 | 1.00 | 1712.95
Validated Improvement | 4 | 0 | 1.00 | 1758.24
>>>
```

Figure 28. Live final comparison of the four variants.

Variant	Correct answers	Incorrect answers	Average attempts	Average latency (ms)
Baseline	4	0	1.00	1760.45
Evaluator Repair	4	0	1.00	1719.77
Lesson Memory	4	0	1.00	1712.95
Validated Improvement	4	0	1.00	1758.24

All four variants answered all four held-out questions correctly on their first attempt. The experiment therefore shows equal held-out accuracy, not an accuracy gain from lessons. The small reference set and deterministic fact checks limit generalization; the evaluator does not establish the correctness of arbitrary natural-language claims.

Verification implementation

verify_hw09.py checks the live Redis index, API, model, authenticated repeat-query hit, frozen labels, record counts, recomputed metrics/costs, stale invalidation, repair, lesson reuse, and unchanged held-out inputs. Its checks evaluate behavior rather than exact answer wording.

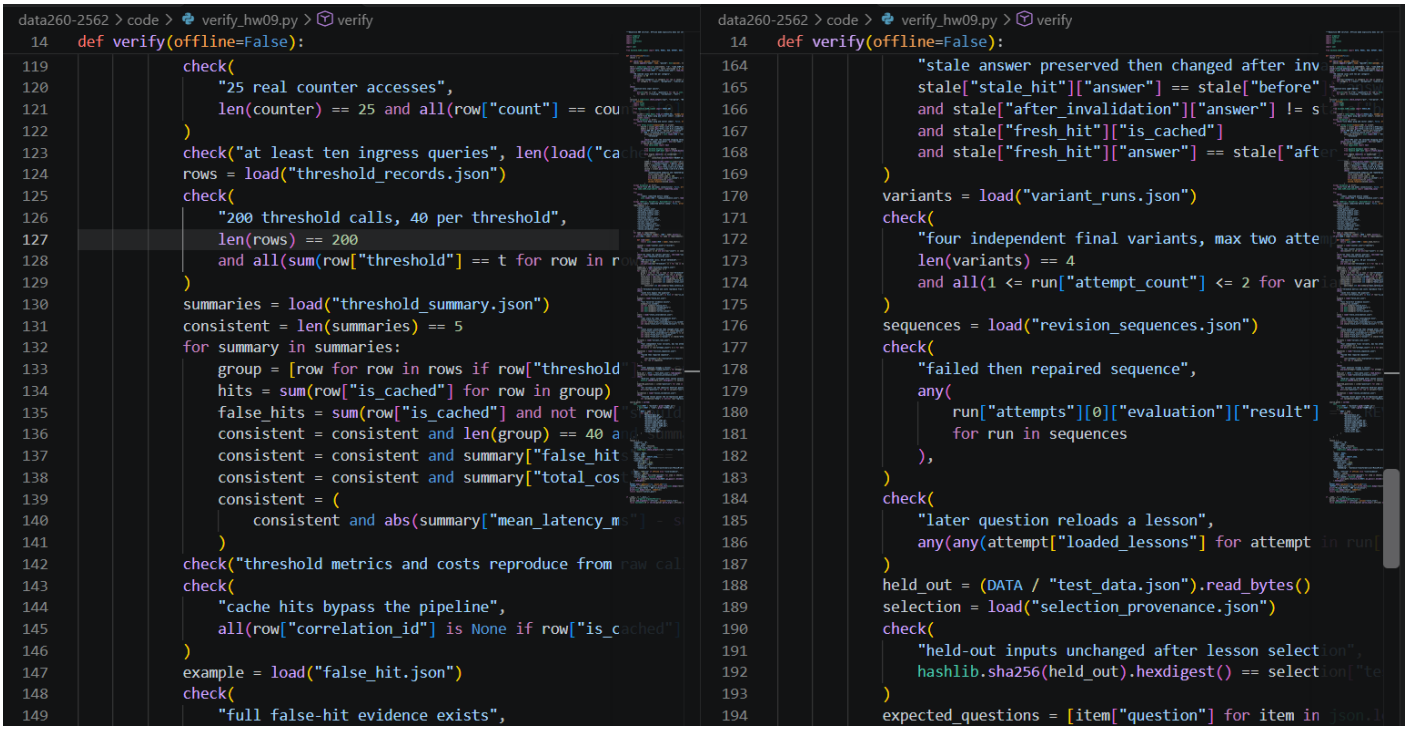


Figure 29. Objective counter, sweep, invalidation, repair, and held-out checks.

Evidence	Verified result
Counter and ingress	25 accesses; 12 ingress queries
Threshold experiment	200 calls, exactly 40 at each of five thresholds
Reproducibility	Hit counts, false-hit counts, costs, and mean latencies match raw data
Agent behavior	Four variants; at most two attempts; lesson reuse and validation checked

The repository evidence is stored in reports/hw09/RUN_LOG.txt, raw/, METRICS.md, AI_USE.md, verification.json, lessons.json, and data/. The frozen cache cases are in the repository-root cache_cases.yaml.

Live verification results

The live run at 2026-11-09T22:13:56.538916+00:00 passed all 36 checks with overall_pass=true. Its offline behavioral suite passed 12 tests. The API was enabled on port 8762 and used the selected threshold of 0.95.

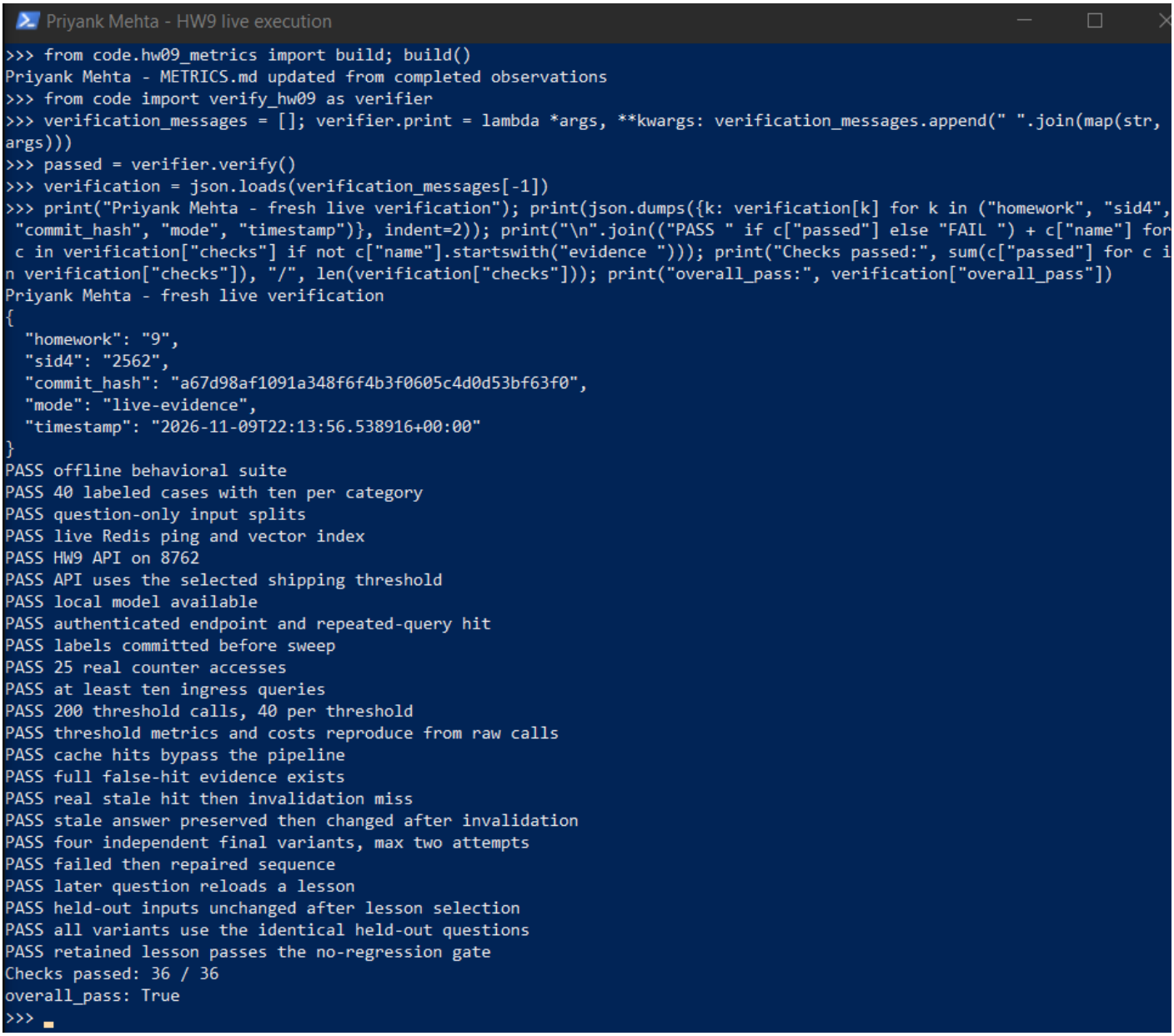


Figure 30. Fresh live verification metadata, passing checks, and overall result.

Verification commit: a67d98af1091a348f6f4b3f0605c4d0d53bf63f0. The tested working tree is additionally identified by the source digest in verification.json.

AI Use

1. What did I use an AI assistant for, and what did I do myself?

I used ChatGPT to troubleshoot installation and command-line issues.

I independently implemented and tested the code and scripts.

2. What AI-produced output was wrong or unsuitable?

It suggested an incorrect command for a python dependency installation.

3. How did I detect the problem or verify the result?

The command resulted in error on execution.

4. What did I change, and why does it work now?

I checked the command through the documentation and executed it, which installed the python dependency successfully.